

CONTROLE DE REEMBOLSO

Documentação Técnica do Backend

Desafio Técnico — Pitang Agile IT

Autora: Aline Santos

github.com/alinec-santos/controle-reembolso-fullstack

1. Visão Geral do Sistema

O Controle de Reembolso é uma aplicação fullstack desenvolvida como desafio técnico para a trilha de desenvolvimento da Pitang Agile IT. O sistema digitaliza e automatiza o fluxo de solicitações de reembolso de despesas corporativas, desde a criação pelo colaborador até o pagamento pelo time financeiro.

Objetivo do sistema

- Colaboradores criam e enviam solicitações de reembolso com descrição, valor, categoria e data da despesa.
- Gestores analisam, aprovam ou rejeitam as solicitações enviadas.
- O time Financeiro confirma o pagamento das solicitações aprovadas.
- Administradores gerenciam usuários e categorias do sistema.

Fluxo principal de uma solicitação

Status	Quem age	Ação
RASCUNHO	Colaborador	Cria a solicitação — pode editar livremente
ENVIADO	Colaborador	Envia para análise — não pode mais editar
APROVADO	Gestor	Aprova a solicitação
REJEITADO	Gestor	Rejeita com justificativa obrigatória
PAGO	Financeiro	Confirma o pagamento
CANCELADO	Colaborador	Cancela quando ainda em RASCUNHO ou ENVIADO

2. Tecnologias Utilizadas

2.1 Backend

Tecnologia	Versão	Função no projeto
Node.js	LTS	Runtime JavaScript no servidor
Express.js	^5.2	Framework HTTP — define rotas, middlewares e controllers
TypeScript	^6.0	Tipagem estática — reduz erros em tempo de desenvolvimento
Prisma ORM	^6.19	Mapeamento objeto-relacional e comunicação com o banco SQLite
SQLite	—	Banco de dados relacional em arquivo — ideal para desenvolvimento e testes
JWT (jsonwebtoken)	^9.0	Geração e validação de tokens de autenticação
Bcrypt	^6.0	Hash seguro de senhas antes de persistir no banco
Zod	^4.4	Validação de schemas e dados de entrada nas requisições
DayJS	^1.11	Manipulação e formatação de datas com suporte a locale pt-BR
Jest + Supertest	^30 / ^7.2	Testes automatizados de integração da API
Docker / Compose	—	Containerização e orquestração do ambiente completo

2.2 Por que essas tecnologias?

- Express 5: versão mais recente, com suporte nativo a async/await nos controllers sem precisar de wrappers.
- Prisma: gera tipos TypeScript automaticamente a partir do schema, tornando as queries type-safe e evitando erros de digitação.
- SQLite: banco embutido em arquivo, sem necessidade de servidor separado — ideal para o contexto de desafio e para rodar testes isolados sem conflito de dados.
- Zod: validação declarativa com mensagens de erro customizadas; integra perfeitamente com TypeScript.

- DayJS: biblioteca leve (2kb) de datas com locale pt-BR, usada para formatar as datas de resposta da API no padrão DD/MM/YYYY.
- JWT: padrão de mercado para autenticação stateless — o servidor não precisa armazenar sessão, basta verificar a assinatura do token.

3. Estrutura de Pastas e Arquivos

3.1 Visão geral do repositório

```
controle-reembolso-fullstack/
├─ backend/           → API REST (Node + Express + TypeScript)
├─ frontend/          → Interface web (React + Vite + TypeScript)
├─ docs/              → Documentação, Postman collection e desafio original
├─ docker-compose.yml → Orquestração dos containers
└─ README.md          → Guia de instalação e uso
```

3.2 Estrutura do backend

```
backend/
├─ src/
│  ├─ @types/          → Extensão de tipos do Express (req.user)
│  ├─ app.ts           → Configuração do Express e registro de rotas
│  ├─ server.ts        → Sobe o servidor na porta 3000
│  ├─ errors/
│  │  └─ AppError.ts   → Classe de erro customizado com statusCode
│  ├─ lib/
│  │  ├─ prisma.ts      → Instância singleton do Prisma Client
│  │  └─ formatDate.ts → Utilitários de data com DayJS
│  ├─ middlewares/
│  │  ├─ auth.middleware.ts      → Verifica JWT e injeta req.user
│  │  ├─ role.middleware.ts      → Verifica perfil do usuário
│  │  ├─ error.middleware.ts     → Captura global de erros
│  │  ├─ validate-params.middleware.ts → Valida parâmetros de URL
│  │  └─ validate-query.middleware.ts → Valida query strings
│  ├─ modules/
│  │  ├─ auth/                → Login e /me
│  │  ├─ category/           → CRUD de categorias
│  │  ├─ reimbursement/      → Fluxo completo de reembolso
│  │  ├─ user/               → Gestão de usuários
│  │  └─ attachment/         → Anexos (URL simulada)
│  ├─ routes/                → Definição de rotas e middlewares por módulo
│  ├─ schemas/               → Schemas Zod compartilhados (params, query)
│  └─ tests/                 → Testes de integração por módulo
├─ prisma/
│  ├─ schema.prisma         → Modelos e enums do banco de dados
│  ├─ migrations/           → Histórico de alterações do banco
│  └─ seed.ts               → Seed com dados iniciais (usuários e categorias)
├─ Dockerfile               → Imagem Docker do backend
├─ jest.config.js           → Configuração do Jest (preset ts-jest)
└─ package.json             → Dependências e scripts
```

3.3 Organização por módulos (padrão modular)

Cada módulo contém seus próprios controllers e schemas. Essa separação facilita a manutenção: para alterar a lógica de reembolso, você trabalha apenas em `src/modules/reimbursement/`, sem impactar outros módulos.

Módulo	Responsabilidade
auth	Autenticação — login e consulta do usuário logado (/me)
reimbursement	Ciclo completo de reembolso: criar, enviar, aprovar, rejeitar, pagar, cancelar
category	Criação, listagem e ativação/inativação de categorias
user	Cadastro, listagem e atualização de usuários (somente ADMIN)
attachment	Registro de anexos simulados via URL (sem upload real de arquivo)

4. Banco de Dados

4.1 Tecnologia e decisão

O banco utilizado é SQLite, gerenciado pelo Prisma ORM. O SQLite armazena tudo em um arquivo dev.db na pasta backend/prisma/, eliminando a necessidade de instalar e configurar um servidor de banco de dados separado. Essa escolha facilita o setup local e os testes automatizados.

Por que SQLite?: Ideal para projetos de demonstração e testes. O banco sobe junto com a aplicação sem configuração extra. Em produção, o Prisma permite migrar facilmente para PostgreSQL ou MySQL alterando apenas uma linha no schema.

4.2 Modelos (schema.prisma)

Model	Campos principais	Relacionamentos
User	id, name, email, password, role, createdAt, updatedAt	Tem muitas ReimbursementRequests e RequestHistories
Category	id, name, active, createdAt, updatedAt	Tem muitas ReimbursementRequests
ReimbursementRequest	id, userId, categoryId, description, amount, expenseDate, status, rejectionReason	Pertence a User e Category; tem Attachments e Histories
Attachment	id, requestId, fileName, fileType, fileUrl, createdAt	Pertence a ReimbursementRequest
RequestHistory	id, requestId, userId, action, observation, createdAt	Pertence a ReimbursementRequest e User

4.3 Enums

Enum	Valores
UserRole	COLABORADOR GESTOR FINANCEIRO ADMIN
RequestStatus	RASCUNHO ENVIADO APROVADO REJEITADO PAGO CANCELADO
RequestAction	CREATED UPDATED SUBMITTED APPROVED REJECTED PAID CANCELED

4.4 Seed

O arquivo `seed.ts` popula o banco com dados iniciais ao rodar `npx prisma db seed` (ou automaticamente no Docker). O seed cria um usuário para cada perfil (ADMIN, GESTOR, FINANCEIRO, COLABORADOR) e algumas categorias padrão, permitindo testar o sistema imediatamente.

5. Autenticação e Autorização

5.1 JWT — Como funciona

A autenticação utiliza JSON Web Token (JWT). O fluxo é:

- 1. O cliente envia email e senha via POST /auth/login.
- 2. O backend valida as credenciais com Zod, busca o usuário no banco e compara a senha com o hash bcrypt.
- 3. Se válido, gera um token JWT assinado com JWT_SECRET, contendo userId e role, com expiração de 1 dia.
- 4. Nas requisições seguintes, o cliente envia o token no header: Authorization: Bearer <token>.
- 5. O authMiddleware intercepta, verifica a assinatura do token e busca o usuário no banco para confirmar que ainda existe.
- 6. O usuário autenticado fica disponível em req.user para todos os controllers.

Decisão de segurança: Mesmo com token válido, o sistema rebusca o usuário no banco (Prisma). Isso garante que um usuário deletado ou desativado não consiga operar — o token sozinho não é suficiente.

5.2 RBAC — Controle de Acesso por Perfil

O roleMiddleware recebe um array de perfis permitidos e verifica se req.user.role está na lista. Se não estiver, retorna 403 Forbidden.

Rota	Método	Perfis autorizados
POST /auth/login	POST	Público
GET /auth/me	GET	Qualquer autenticado
POST /users	POST	ADMIN
GET /users	GET	ADMIN
POST /categories	POST	ADMIN
PUT /categories/:id	PUT	ADMIN
GET /categories	GET	Qualquer autenticado
POST /reimbursements	POST	COLABORADOR
GET /reimbursements	GET	Qualquer autenticado (filtros por perfil)

Rota	Método	Perfis autorizados
POST /reimbursements/:id/submit	POST	COLABORADOR (somente dono)
PUT /reimbursements/:id	PUT	COLABORADOR (somente dono, status RASCUNHO)
POST /reimbursements/:id/approve	POST	GESTOR
POST /reimbursements/:id/reject	POST	GESTOR
POST /reimbursements/:id/pay	POST	FINANCEIRO
POST /reimbursements/:id/cancel	POST	COLABORADOR (somente dono)

6. Regras de Negócio e Validações

6.1 Validação de entrada com Zod

Antes de qualquer lógica de negócio, os dados de entrada são validados com Zod. Caso a validação falhe, o `errorMiddleware` captura o `ZodError` e retorna 400 com a mensagem do primeiro erro encontrado.

Schema	Campos validados
loginSchema	email (string email válido), password (mínimo 6 caracteres)
createRequestSchema	categoryId (string), description (mínimo 3 chars), amount (número positivo), expenseDate (string)
rejectRequestSchema	reason (obrigatório — gestor deve justificar a rejeição)
createUserSchema	name (mínimo 3 chars), email (válido), password (mínimo 6), role (enum UserRole)
idParamsSchema	id (UUID válido — impede IDs malformados de chegar ao banco)
categorySchema	name (obrigatório), active (boolean — opcional no update)

6.2 Regras de transição de status

Cada ação verifica o status atual antes de aplicar a transição. Tentar executar uma ação em status inválido retorna 400.

Ação	Status exigido	Erro se diferente
submit (enviar)	RASCUNHO	"Apenas solicitações em RASCUNHO podem ser enviadas"
update (editar)	RASCUNHO	"Apenas solicitações em RASCUNHO podem ser editadas"
approve (aprovar)	ENVIADO	"Apenas solicitações ENVIADAS podem ser aprovadas"
reject (rejeitar)	ENVIADO	"Apenas solicitações ENVIADAS podem ser rejeitadas"

Ação	Status exigido	Erro se diferente
pay (pagar)	APROVADO	"Apenas solicitações APROVADAS podem ser pagas"
cancel (cancelar)	RASCUNHO ou ENVIADO	"Só é possível cancelar em RASCUNHO ou ENVIADO"

6.3 Validações de propriedade

- Somente o dono da solicitação (userId === req.user.id) pode enviá-la, editá-la ou cancelá-la.
- Gestor não pode aprovar/rejeitar solicitação que não esteja no status ENVIADO.
- Financeiro não pode pagar solicitação que não esteja APROVADA.
- Colaborador não consegue ver solicitações de outro colaborador — o controller filtra por userId.

6.4 Regras de categoria

- Categorias têm o campo active (boolean). Ao criar uma solicitação, o sistema verifica se a categoria existe e está ativa.
- Apenas ADMIN pode criar ou atualizar categorias.
- Inativar uma categoria não afeta solicitações já criadas.

6.5 Listagem filtrada por perfil

O GET /reimbursements adapta o filtro conforme o perfil do usuário autenticado:

Perfil	O que vê
COLABORADOR	Apenas suas próprias solicitações (todas os status)
GESTOR	Solicitações com status ENVIADO, APROVADO ou REJEITADO
FINANCEIRO	Solicitações com status APROVADO ou PAGO
ADMIN	Todas as solicitações sem filtro de status

6.6 Trilha de auditoria (histórico)

Toda ação relevante gera um registro em RequestHistory contendo: o ID da solicitação, o ID do usuário que executou a ação, o tipo da ação (enum RequestAction) e uma observação descritiva. Isso permite rastrear quem fez o quê e quando em toda a vida da solicitação.

7. Middlewares

Middleware	Função	Retorno em caso de falha
authMiddleware	Verifica o Bearer Token JWT no header. Se válido, busca o usuário no banco e injeta em req.user.	401 Unauthorized
roleMiddleware(roles)	Verifica se req.user.role está entre os perfis permitidos.	403 Forbidden
validateParams(schema)	Valida os parâmetros de URL (ex: :id UUID) com Zod.	400 Bad Request via errorMiddleware
validateQuery(schema)	Valida query strings da URL com Zod.	400 Bad Request via errorMiddleware
errorMiddleware	Middleware global registrado no final do app.ts. Captura AppError (retorna o statusCode definido), ZodError (retorna 400) e erros genéricos (retorna 500).	Varia conforme o tipo

Ordem importa!: No Express, middlewares são executados na ordem em que são registrados. O errorMiddleware DEVE ser o último a ser registrado (app.use(errorMiddleware)) para capturar erros de todas as rotas.

8. Endpoints da API

8.1 Auth

Método	Rota	Descrição	Auth
POST	/auth/login	Autentica usuário e retorna token JWT + dados do usuário	Não
GET	/auth/me	Retorna dados do usuário autenticado	Sim

8.2 Users

Método	Rota	Descrição	Perfil
POST	/users	Cria novo usuário	ADMIN
GET	/users	Lista todos os usuários	ADMIN
PUT	/users/:id	Atualiza dados de um usuário	ADMIN

8.3 Categories

Método	Rota	Descrição	Perfil
GET	/categories	Lista todas as categorias	Autenticado
POST	/categories	Cria nova categoria	ADMIN
PUT	/categories/:id	Atualiza nome ou status da categoria	ADMIN

8.4 Reimbursements

Método	Rota	Descrição	Perfil
POST	/reimbursements	Cria solicitação (status RASCUNHO)	COLABORADOR

Método	Rota	Descrição	Perfil
GET	/reimbursements	Lista solicitações (filtrado por perfil)	Autenticado
GET	/reimbursements/:id	Detalha uma solicitação específica	Autenticado
PUT	/reimbursements/:id	Edita solicitação (somente RASCUNHO, somente dono)	COLABORADOR
POST	/reimbursements/:id/submit	Envia para análise	COLABORADOR (dono)
POST	/reimbursements/:id/approve	Aprova solicitação	GESTOR
POST	/reimbursements/:id/reject	Rejeita com justificativa obrigatória	GESTOR
POST	/reimbursements/:id/pay	Marca como paga	FINANCEIRO
POST	/reimbursements/:id/cancel	Cancela solicitação	COLABORADOR (dono)
GET	/reimbursements/:id/history	Lista histórico de ações	Autenticado
POST	/reimbursements/:id/attachments	Adiciona anexo (via URL)	Autenticado
GET	/reimbursements/:id/attachments	Lista anexos da solicitação	Autenticado

9. Testes Automatizados

9.1 Ferramentas utilizadas

Ferramenta	Função
Jest	Framework de testes — organiza describes e its, executa assertions
Supertest	Simula requisições HTTP reais contra a aplicação Express sem subir um servidor
ts-jest	Preset que permite executar arquivos TypeScript diretamente no Jest sem build prévio
Prisma Client	Usado nos testes para setup (beforeEach) e teardown (afterAll) do banco SQLite

9.2 Estratégia de testes

Os testes são de integração: cada teste faz requisições HTTP reais contra o app Express, que por sua vez acessa o banco SQLite de desenvolvimento. O `jest.config.js` usa `--runInBand` para rodar os testes em série (um de cada vez), evitando conflitos de dados no banco compartilhado.

- `beforeEach`: limpa todas as tabelas e recria os dados necessários para o teste (usuários, categorias). Isso garante isolamento entre testes.
- `afterAll`: desconecta o Prisma para evitar conexões abertas após o fim da suíte.

9.3 O que está sendo testado

Arquivo de teste	Cenários cobertos
<code>auth.test.ts</code>	Login com credenciais válidas; login com senha errada (401)
<code>user.test.ts</code>	Criação de usuário pelo ADMIN; tentativa de criação por não-ADMIN (403); email duplicado (409)
<code>category.test.ts</code>	Listagem; criação pelo ADMIN; criação por COLABORADOR (403); atualização; acesso sem autenticação (401)
<code>reimbursement.test.ts</code>	Criação de rascunho; listagem filtrada por perfil; envio; tentativa de enviar duas vezes; aprovação; rejeição com justificativa; pagamento; histórico;

Arquivo de teste	Cenários cobertos
	rejeição sem justificativa (400); pagamento sem aprovação (400); categoria inativa (400); colaborador acessando solicitação de outro (403)

9.4 Como executar

```
cd backend  
npm test
```

Atenção: Ao rodar os testes, o banco SQLite dev.db será limpo (deleteMany em todas as tabelas no beforeEach). Se estiver usando o banco para desenvolvimento manual, rode npx prisma db seed novamente após os testes.

9.5 Possíveis melhorias nos testes

- Banco de teste separado: configurar DATABASE_URL apontando para um arquivo test.db exclusivo, evitando sobrescrever dados de desenvolvimento.
- Cobertura de código: adicionar flag --coverage no Jest para gerar relatório de quais linhas estão cobertas.
- Testes unitários: criar testes isolados para funções utilitárias (formatDate, parseDate) e schemas Zod.
- Testes de validação de UUID: testar idParamsSchema com IDs malformados para garantir o retorno 400.
- Mock do Prisma: em testes unitários de controllers, usar jest.mock para simular o banco e testar apenas a lógica sem dependência de I/O.

10. Docker e DevOps

O projeto possui um `docker-compose.yml` que orquestra dois serviços: backend (Node + Express) e frontend (React + Vite). Ao rodar `docker compose up --build`, o Docker:

- Instala as dependências do backend e frontend.
- Executa as migrations do Prisma (`npx prisma migrate deploy`).
- Popula o banco com o seed (`npx prisma db seed`).
- Sobe o backend na porta 3000 e o frontend na porta 5173.

Vantagem: Com Docker, qualquer pessoa consegue rodar o projeto completo com um único comando, sem precisar instalar Node, SQLite ou configurar variáveis manualmente.

Serviço	Porta	URL de acesso
Backend (API)	3000	http://localhost:3000
Frontend (UI)	5173	http://localhost:5173

11. Ferramentas de IA no Desenvolvimento

O projeto utilizou diferentes ferramentas de IA em etapas específicas do desenvolvimento, cada uma contribuindo com sua especialidade:

11.1 Organização e estruturação — ChatGPT e Gemini

ChatGPT e Gemini foram utilizados nas fases iniciais de planejamento e estruturação do projeto.

- Definição do escopo do sistema: quais funcionalidades implementar, quais deixar de fora para o MVP.
- Estruturação do fluxo de reembolso: levantamento dos status possíveis e as transições entre eles.
- Organização do backlog de tarefas e ordem de implementação.
- Revisão do schema do banco: ajudaram a identificar quais campos eram essenciais e quais relacionamentos eram necessários.
- Geração de perguntas para simular entrevistas técnicas sobre o projeto.

Papel: Funcionaram como 'parceiros de brainstorming' para tomar decisões de arquitetura e priorizar o que implementar dentro do tempo disponível do desafio.

11.2 Desenvolvimento de código — Claude e Antigravity

Claude (Anthropic) e Antigravity foram as principais ferramentas de geração e revisão de código durante o desenvolvimento.

- Claude: geração dos controllers de reembolso com as regras de negócio embutidas (validações de status, verificação de propriedade). Revisão de middlewares e sugestão de melhorias de segurança no authMiddleware, como a verificação do usuário no banco mesmo com token válido.
- Claude: suporte na escrita dos testes de integração com Jest + Supertest, especialmente nos fluxos negativos (tentativas de operações inválidas).
- Antigravity: auxílio no desenvolvimento de partes específicas do código, sugestões de refatoração e geração de código boilerplate para novos módulos.

Papel: Atuaram como pair programmers — acelerando a escrita de código repetitivo e ajudando a pensar nos casos de borda das regras de negócio.

11.3 Layout e UI — Antigravity e Banani

As ferramentas de IA focadas em interface foram utilizadas para o frontend do projeto.

- Antigravity: geração de componentes React com Tailwind CSS, estruturação das páginas e sugestão de layouts para formulários e tabelas de listagem.
- Banani: ferramenta especializada em UI/UX, utilizada para gerar protótipos e sugestões visuais das telas do sistema (dashboard, formulário de solicitação, tela de aprovação).

Papel: Reduziram o tempo de desenvolvimento do frontend ao gerar código de componentes e estruturas visuais que só precisaram de pequenos ajustes.

11.4 Testes — ferramentas e como poderiam ter auxiliado

Os testes foram desenvolvidos principalmente manualmente com apoio de Claude para os fluxos mais complexos. Outras ferramentas de IA poderiam ter contribuído ainda mais:

- GitHub Copilot: geração automática de casos de teste a partir dos controllers — ao escrever `it('deve...')`, o Copilot sugere o corpo completo do teste com base no contexto do arquivo.
- ChatGPT / Claude: úteis para listar todos os casos de borda de uma regra de negócio ('quais cenários de erro devo testar no endpoint de aprovação?'), garantindo cobertura mais ampla.
- Keploy / Orion: ferramentas especializadas em geração automática de testes de API — poderiam ter gravado as requisições do Postman e gerado os testes Jest automaticamente.
- Testim ou Playwright AI: para testes end-to-end no frontend, poderiam ter gravado os fluxos de usuário e gerado scripts de automação.

12. Funcionalidades Extras Implementadas

Além do fluxo básico exigido pelo desafio, o projeto inclui funcionalidades adicionais que demonstram cuidado com qualidade e segurança:

12.1 Histórico de auditoria completo

Toda ação executada em uma solicitação gera um registro em RequestHistory. Isso permite rastrear o ciclo de vida completo de cada solicitação, identificar quem executou cada ação e quando, e facilitar auditorias internas.

12.2 Validação de UUID nos parâmetros de URL

O middleware validateParams com o schema idParamsSchema (z.string().uuid()) valida que o :id passado na URL é um UUID válido antes de qualquer consulta ao banco. Isso previne erros de banco por IDs malformados e pode bloquear tentativas de injection básicas.

12.3 Verificação do usuário no banco após validação do JWT

O authMiddleware não confia apenas na assinatura do token. Mesmo com token válido, ele rebusca o usuário no banco de dados. Isso garante que usuários deletados ou suspensos não consigam operar com tokens ainda válidos.

12.4 Filtragem de dados sensíveis

Ao retornar dados de usuário (login, listagem, criação), o campo password nunca é incluído na resposta. O Prisma select é usado explicitamente para garantir que apenas os campos necessários sejam retornados.

12.5 Formatação de datas com DayJS e locale pt-BR

Todas as datas retornadas pela API são formatadas no padrão brasileiro (DD/MM/YYYY ou DD/MM/YYYY HH:mm) usando DayJS com locale pt-BR. Isso evita que o frontend precise converter datas ISO para o formato local.

12.6 Tratamento centralizado de erros

O `errorMiddleware` centraliza o tratamento de todos os tipos de erro da aplicação. Os controllers simplesmente lançam `AppError` ou deixam o `ZodError` propagar — o middleware cuida do formato da resposta. Isso elimina duplicação de código de tratamento de erro em cada controller.

12.7 Collection Postman documentada

A collection do Postman em `docs/postman/` inclui todos os endpoints com scripts automáticos que salvam o token JWT após o login, permitindo testar o fluxo completo sem configuração manual. Isso facilita tanto a revisão do avaliador quanto os testes manuais durante o desenvolvimento.

12.8 Docker com seed automático

O `docker-compose.yml` executa migrations e seed automaticamente ao subir o ambiente. Qualquer pessoa consegue rodar o projeto completo com `docker compose up --build` sem nenhuma configuração adicional.

13. Perguntas Frequentes em Entrevistas

Sobre autenticação

Por que você verifica o usuário no banco mesmo após validar o JWT?

Porque o token tem validade de 1 dia. Se um usuário for deletado ou suspenso nesse intervalo, o token ainda seria tecnicamente válido. Rebuscar no banco garante que apenas usuários ativos consigam operar.

O que é RBAC e como você implementou?

Role-Based Access Control — controle de acesso baseado em perfil. Implementei com o `roleMiddleware`, que recebe um array de perfis permitidos e retorna 403 se o perfil do usuário autenticado (em `req.user.role`) não estiver na lista.

Sobre validações

Por que usar Zod se o TypeScript já tem tipagem?

TypeScript valida em tempo de compilação, mas dados externos (corpo da requisição, parâmetros de URL) chegam como `any` em runtime. Zod valida em runtime, garantindo que os dados realmente correspondam ao tipo esperado antes de chegar na lógica de negócio.

Como você garante que o `:id` da URL é válido antes de consultar o banco?

Uso o middleware `validateParams` com o schema `idParamsSchema` (`z.string().uuid()`). Se o id não for um UUID válido, o Zod lança um erro que é capturado pelo `errorMiddleware` e retorna 400 antes de qualquer acesso ao banco.

Sobre o banco de dados

Por que SQLite e não PostgreSQL?

Para um projeto de demonstração/desafio, SQLite elimina a necessidade de configurar um servidor de banco separado. O Prisma abstrai o banco, então migrar para PostgreSQL em produção exigiria apenas alterar o provider e a `DATABASE_URL` no schema.

Sobre os testes

Por que `--runInBand` no Jest?

Os testes compartilham o mesmo banco SQLite. Se rodassem em paralelo, um teste poderia deletar dados que outro está usando. O `--runInBand` força execução serial, garantindo isolamento entre suítes.

Por que usar beforeEach para limpar o banco em vez de beforeAll?

Para garantir isolamento total entre cada teste. Com beforeEach, cada it começa com um banco limpo e dados conhecidos, eliminando dependência de ordem de execução.